CS 645

Security and Privacy in Computer Systems

Lecture 5: Application Program Security

- Buffer overflow exploits
- More effective buffer overflow attacks
- Preventing buffer overflow attacks

Announcement

- Project 1 has been posted on the course website
  - Due in 2 weeks, on Feb 28 @ 6pm
  - You can work in teams of up to three students
  - Submit project in Canvas (one submission per team)

Last week

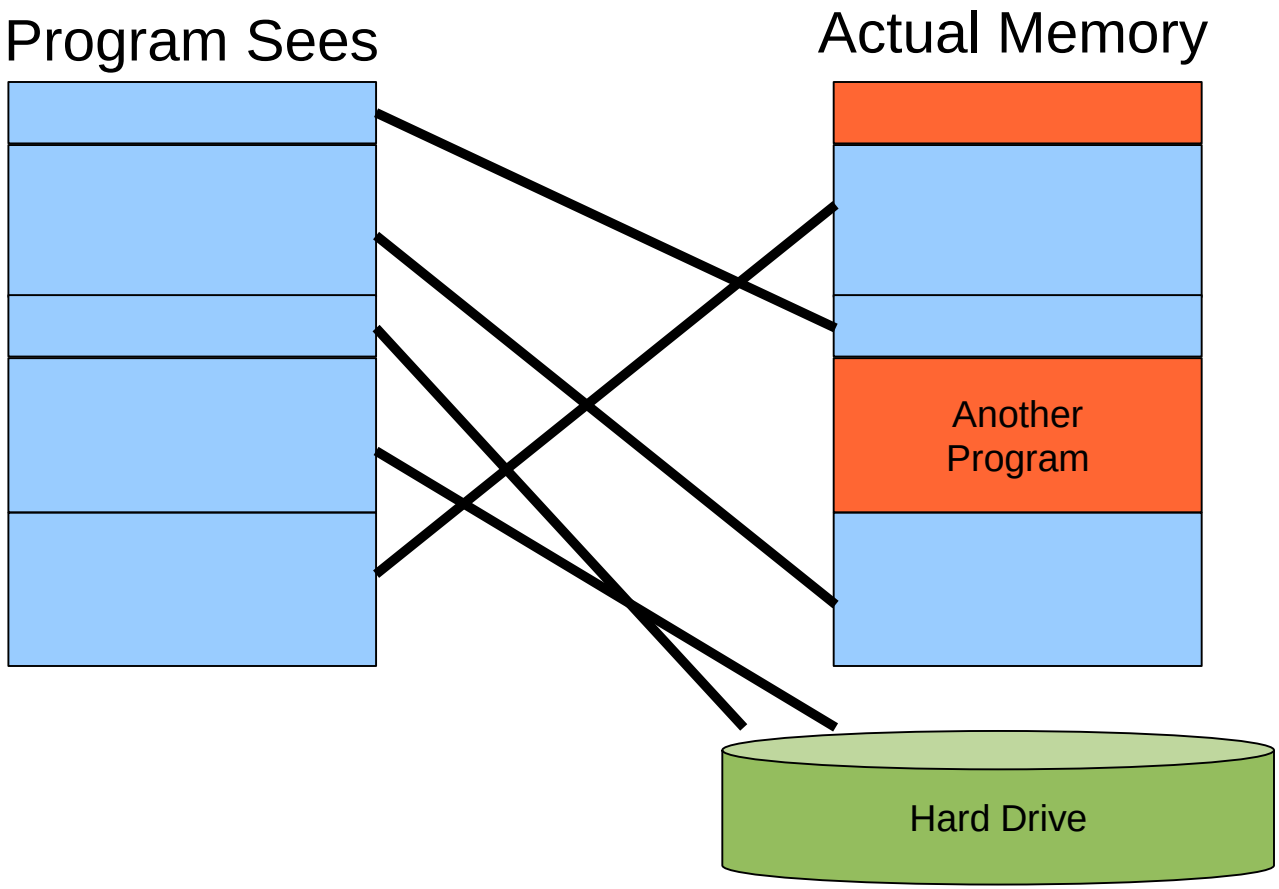
- OS security
- Filesystem security

Buffer Overflow Exploits

Compiling and Linking

- A program source code (e.g., Java or C++) is transformed into machine code instructions through a process called ***compiling***
- A program can be compiled to be statically linked or dynamically linked
- ***Static linking***: all shared libraries needed by a program during its execution are copied into the compiled program on disk
  - Drawback: requires more space on disk, less flexibility
- ***Dynamic linking***: shared libraries are actually loaded when the program is actually run
  - E.g., Windows (DLL – dynamic linking library), UNIX (shared objects)
  - Saves space on disk, allows better modularization (recompile just one DLL which is used by many programs)

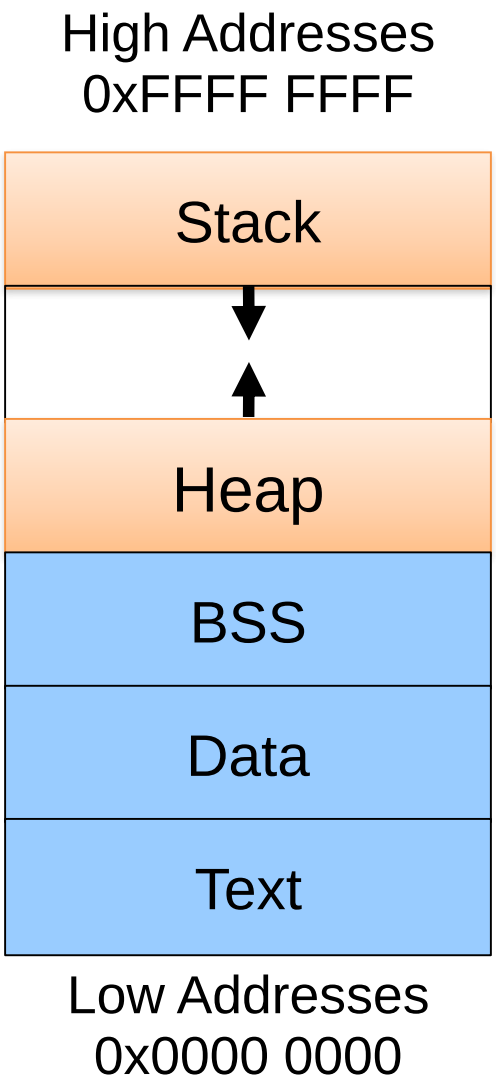
Virtual Memory



Mapping virtual addresses to real addresses

Unix Address Space

- Text: machine code of the program, compiled from the source code
- Data: static program variables initialized in the source code prior to execution
- BSS (block started by symbol): static variables that are uninitialized
- Heap : data dynamically generated during the execution of a process
- Stack: structure that grows downwards and keeps track of the activated method calls, their arguments and local variables



What is an Exploit?

- An exploit is any input (i.e., a piece of software, an argument string, or sequence of commands) that takes advantage of a bug, glitch or vulnerability in order to cause an attack
- An attack is an unintended or unanticipated behavior that occurs on computer software, hardware, or something electronic and that brings an advantage to the attacker

Buffer overflow exploits

- Most common cause of Internet attacks
  - Over 50% of advisories published by CERT (computer security incident report team) are caused by various buffer overflows
- Morris worm (1988): overflow in fingerd
  - Infected 10% of the existing Internet
- CodeRed (2001): overflow in MS-IIS server
  - 300,000 machines infected in 14 hours
- SQL Slammer (2003): overflow in MS-SQL server
  - 75,000 machines infected in **10 minutes (!!)**

Simple Integer Overflow Attacks

- Exploits the way integers are represented in memory

- Signed integers are usually are expressed in two's complement notation
  - positive value → binary representation of the number
    - If greater than 127, an additional byte of 0's is added

    example: 255 → 00000000 11111111
  - negative value:

    1) binary representation of absolute value

    2) invert binary bits

    3) add 1

    example: -5 → 0000 0101 → 1111 1010 → 1111 1011

Simple Integer Overflow Attacks

- Thus, positive integers are in the range 0x00000000 to 0x7FFFFFFF (which is 2^{31} -1), and negative integers are in the range 0x80000000 to 0xFFFFFFFF
- If a program continues to add very large positive integers and exceeds the maximum value, the sum will overflow and become a negative number!
- If many large negative integers are added, the sum may underflow and become positive!

- Also, for unsigned integers (only positive numbers, represented from 0x00000000 to 0xFFFFFFFF), the sum of many large numbers will wrap around to zero.

Example of integer overflow vulnerability

```
#include <stdio.h>

int main(int argc, char *argv[])
{
  unsigned int connections = 0;
  // insert network code here

  // Does nothing to check overflow conditions
  connections++;
  if(connections < 5)
    grant_access();
  else deny_access();

  return 1;

}
```

Example of integer overflow vulnerability - fixed

```
#include <stdio.h>

int main(int argc, char *argv[])
{
  unsigned int connections = 0;
  // insert network code here

  // prevents overflow conditions
  if(connections < 5)
    connections ++;

  if(connections < 5)
    grant_access();
  else deny_access();

  return 1;

}
```

Buffer Overflow Attack

- One of the most common OS bugs is a buffer overflow
  - The developer fails to include code that checks whether an input string fits into its buffer array
  - An input to the running process exceeds the length of the buffer
  - The input string overwrites a portion of the memory of the process
  - Causes the application to behave improperly and unexpectedly
- Effect of a buffer overflow
  - The process can operate on malicious data or execute malicious code passed in by the attacker
  - If the process is executed as root, the malicious code will be executing with root privileges

Buffer Overflow Attack in a Nutshell

- First described in

  Smashing The Stack For Fun And Profit.

  by Aleph One, e-zine www.Phrack.org #49, 1996

  http://insecure.org/stf/smashstack.html

  (recommend reading this!)
- The attacker exploits an unchecked buffer to perform a buffer overflow attack
- The ultimate goal for the attacker is getting a shell that allows to execute arbitrary commands with high privileges
- Kinds of buffer overflow attacks:
  - Stack smashing
  - Heap smashing

Buffer Overflow Attacks

- ***Stack*** is a data structure that uses the Last In First Out (***LIFO***) principle
  - PUSH: add an element to the top of the stack
  - POP: removes an element from the top of the stack

D
B
A

PUSH A
PUSH B
PUSH C
POP
PUSH D

  - Another popular structure is a ***queue*** (***FIFO*** – first in first out)

Buffer Overflow Attacks

- In the context of a program's address space, the ***stack*** is a memory segment that consists of frames, each associated with an active function call
  - Each frame stores:
    - The local variables in the called function
    - Arguments for the function call
    - The return address for the parent call
  - Stack grows downwards in memory
  - At the base of the stack is the frame for the main() call
  - At the top of the stack is the frame for the currently running call

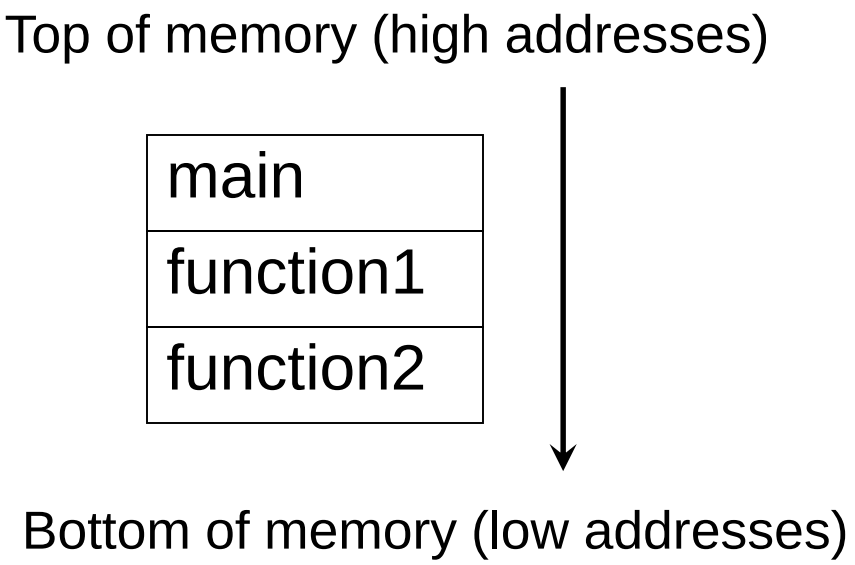
Buffer Overflow Attacks

- Example:

  main

    function1

      function2



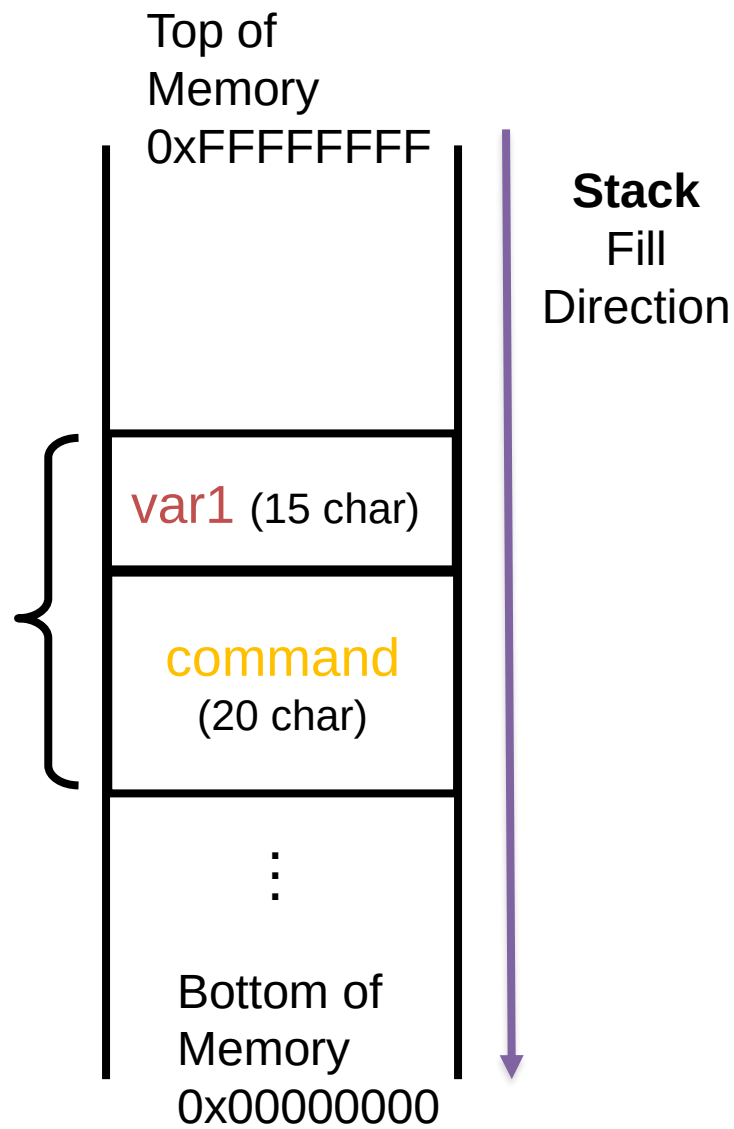
Buffer Overflow

```
domain.c
main(int argc, char *argv[ ])
/* get user_input */
{
    char var1[15];
    char command[20];
    strcpy(command, “whois ");
    strcat(command, argv[1]);
    strcpy(var1, argv[1]);
    printf(var1);
    system(command);
}
```

- Retrieves domain registration info
- e.g., run “domain njit.edu”
- argv[1] is the user input
- strcpy(dest, src) does not check buffer
- strcat(d, s) concatenates strings

Buffer Overflow

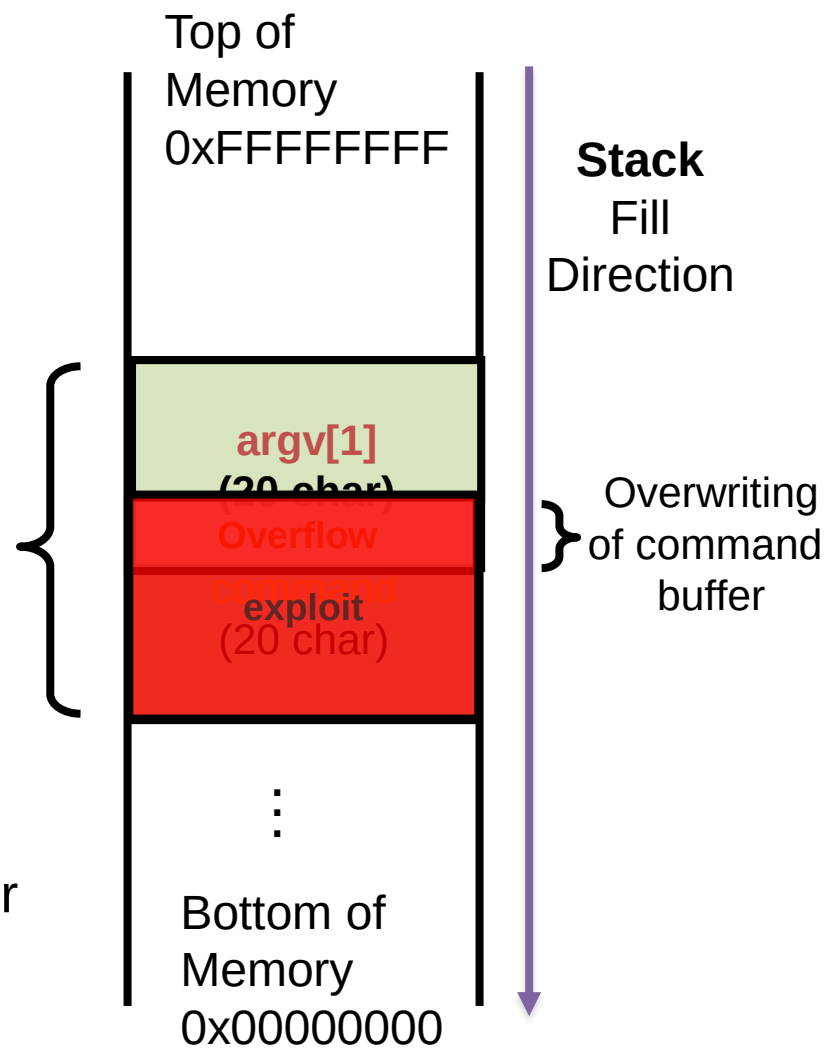
```
domain.c
main(int argc, char *argv[ ])
/* get user_input */
{
    char var1[15];
    char command[20];
    strcpy(command, “whois ");
    strcat(command, argv[1]);
    strcpy(var1, argv[1]);
    printf(var1);
    system(command);
}
```



- Retrieves domain registration info
- e.g., run “domain njit.edu”

strcpy() Vulnerability

```
domain.c
Main(int argc, char *argv[])
/*get user_input*/
{
    char var1[15];
    char command[20];
    strcpy(command, “whois ");
    strcat(command, argv[1]);
    strcpy(var1, argv[1]);
    printf(var1);
    system(command);
}
```



- argv[1] is the user input
- strcpy(dest, src) does not check buffer
- strcat(d, s) concatenates strings

What is the problem?

- In a buffer overflow attack, an attacker provides input that the program blindly copies to a buffer that is smaller than the input
- For a local variable on the stack, a buffer overflow will cause the overwrite of the memory beyond the buffer's allocated space on the stack
  - In the previous example, the attacker overwrites local variables adjacent in memory to the buffer

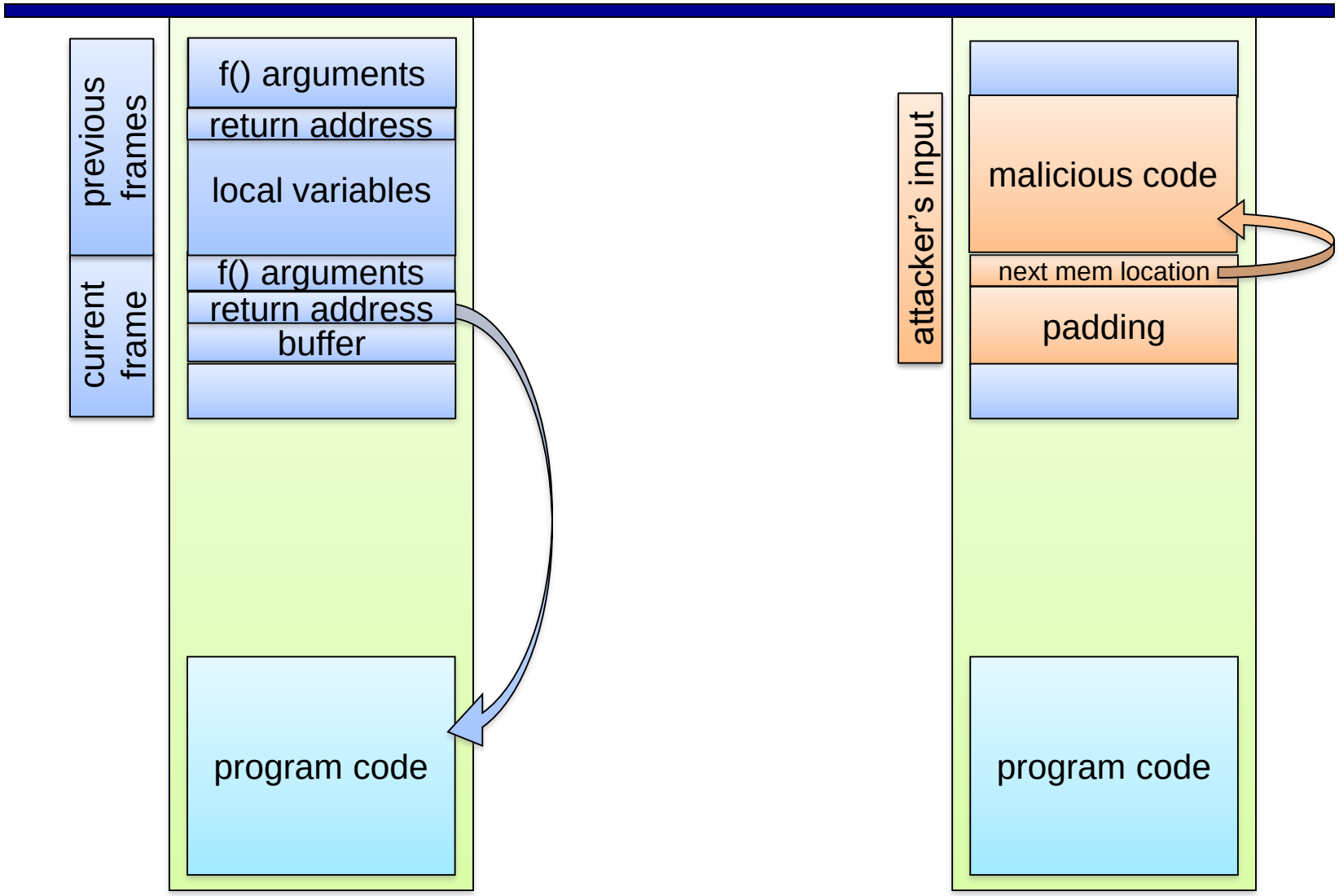
strcpy() vs. strncpy()

- Function strcpy() copies the string in the second argument into the first argument
  - e.g., strcpy(dest, src)
  - If source string > destination string, the overflow characters may occupy the memory space used by other variables
  - The null character is appended at the end of “dest” automatically
- Function strncpy() copies the string by specifying the number **n** of characters to copy
  - e.g., strncpy(dest, src, n); dest[n] = ‘\0’
  - If source string is longer than n characters, the overflow characters are discarded automatically
  - You have to place the null character manually

Problem: no range checking

- strcpy does <u>not</u> check input size
  - strcpy(dest, src) simply copies memory contents into dest starting from *src until "\0" is encountered, ignoring the size of area allocated to dest
- Many C library functions are unsafe
  - strcpy(char *dest, const char *src)
  - strcat(char *dest, const char *src)
  - gets(char *s)
  - scanf(const char *format, …)
  - printf(const char *format, …)

Stack smashing attack



Before the attack

After the attack

Buffer Overflow Issues

- Attacker must first guess the location of the return address with respect to the buffer
- Attacker must determine what address to use for overwriting the return address so that the execution is passed to the attacker's code

- These two are made difficult by the nature of OS design
  - Processes cannot access the address space of other processes (so, malicious code must reside in the address space of the exploited process, usually right in the buffer itself)
  - Address space of a given process is unpredictable, may change when program runs on different machines
    - The value in the return address must point to the beginning of attack assembly code in the buffer
    - Otherwise application will crash with segmentation violation
    - Attacker must correctly guess in which stack position his buffer will be when the function is called

Stack smashing attack

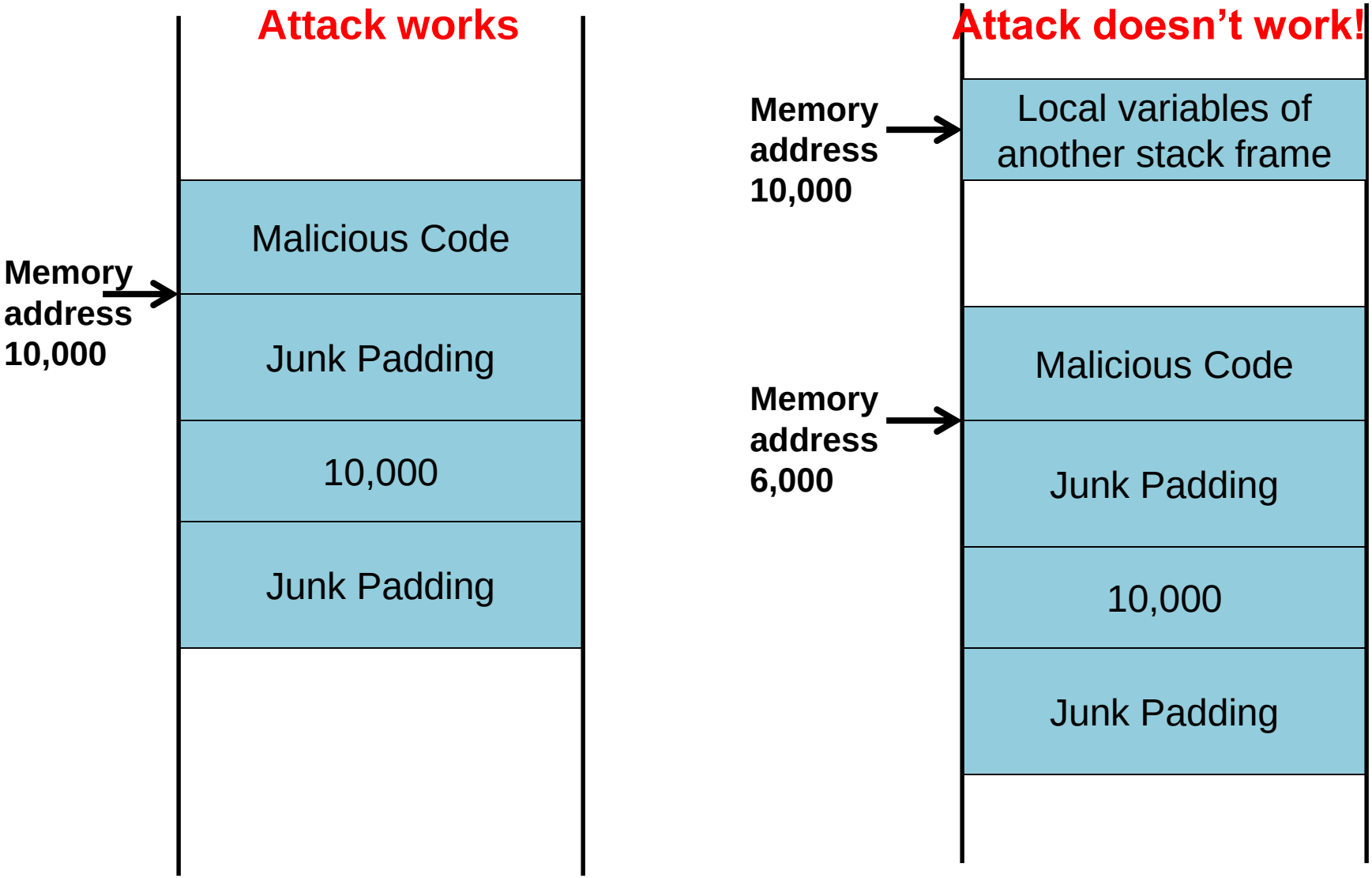
- Buffer contains attacker-created string
  - Attacker puts actual assembly instructions into his input string, e.g.,binary code of execve(“/bin/sh”)
- When function exits, code in the buffer will be executed, giving attacker a shell
  - Root shell if the victim program is setuid root

Buffer Overflow Issues

Consider the following attack buffer:

Malicious Code
Junk Padding
Guessed address of malicious code
Junk Padding

Buffer Overflow Issues

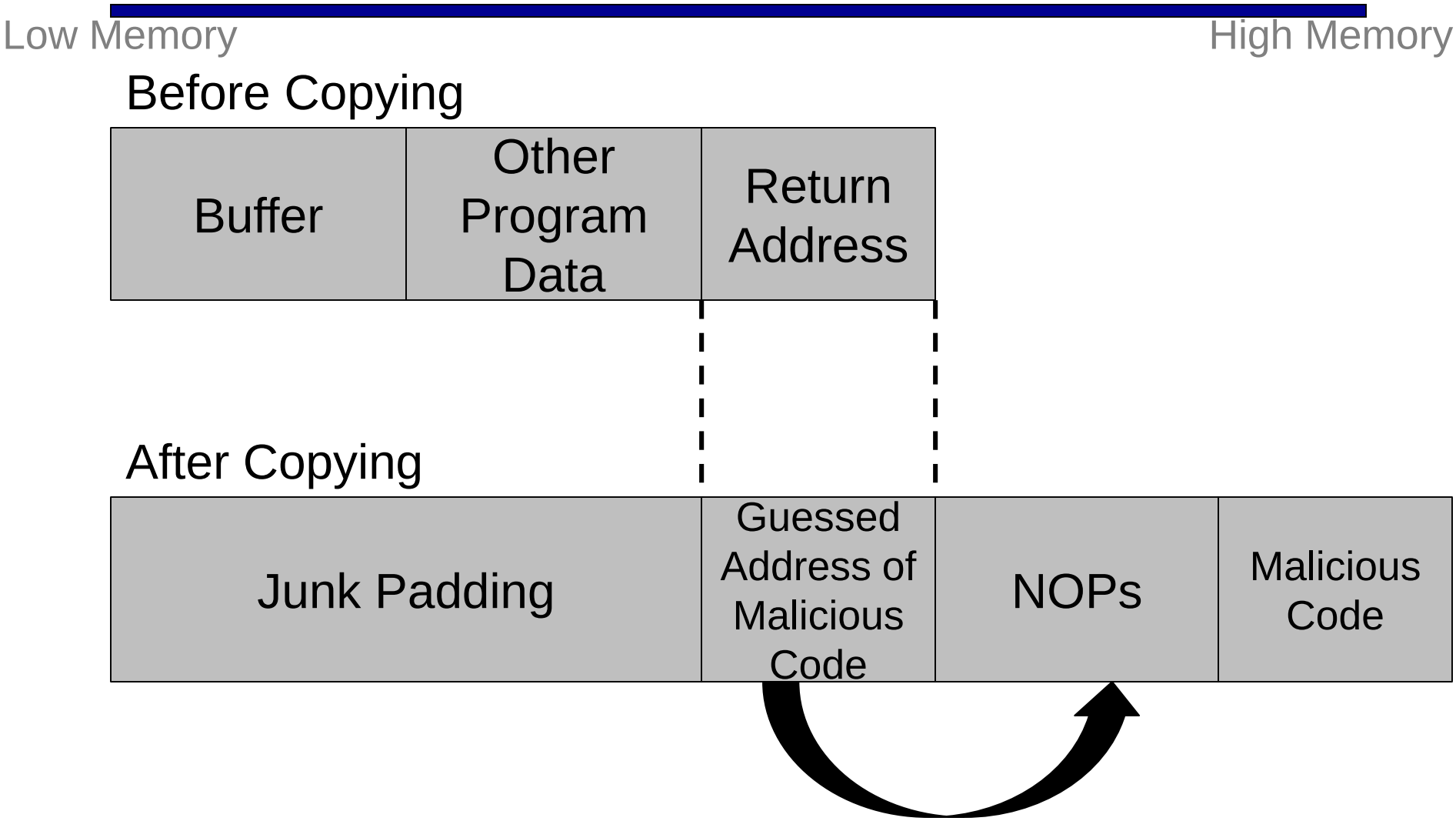


More effective buffer overflow attacks

NOP Sledding

- Increases the attacker's chances to correctly guess the location of the malicious code in memory by increasing the size of the target
- NOP (No-op) = a CPU instruction that does nothing except tell the CPU to go to the next instruction

NOP Sledding



NOP Sledding

- The attacker crafts a payload that contains:
  - enough data to overflow the buffer
  - a guess for a reasonable return address in the process's address space
  - a very large number of NOP instructions (NOP sled)
  - the malicious code
- Once the processor jumps somewhere in the high number of NOPs, the processor will "sled through" all the NOPs until it finally reaches the malicious code

Jump-to-register attack

- NOP sledding may still require a good deal of guesswork
- Processes load external libraries into a reserved portion of their memory address space
  - Thus, memory address of library is predictable

Jump-to-register attack

- Assume a Windows system DLL contains an instruction that tells the processor to jump to the address stored in one of the processor's registers, such as the ESP register: jmp ESP
- Attacker manages to place the malicious code at the address pointed by that register (ESP)
- Attacker then overwrites the RET address of the current function with the address of this known instruction (by overflowing the buffer)
- Then, on return from the function, the processor will execute the "jmp ESP" instruction and thus jump to the address where the malicious code is stored

Return-to-libc attack

- To protect against buffer overflow attacks, the OS can mark the stack as non-executable
  - Malicious code loaded on the stack cannot be executed
- Attacker calls functions from the libc library
  - The libc library is usually loaded with most programs
- How does the attack work?
  - Overwrite stack using the vulnerable buffer
  - Change return address to the system() call within libc
  - Setup the argument to system() as “/bin/bash” on the stack
  - Ensure that system() exits gracefully

Return-to-libc attack example

Low Memory High Memory

Before Copying

Buffer	Other Program Data	Return Address

After Copying

Junk Padding	Address of system()	Address of exit()	Pointer to “/bin/bash” string

Return-to-libc attack example

- The return address now points to the system() function (thus, system() will be executed after the current function returns)
- The next value on stack is assumed to be the return address when system() returns
  - When system() returns, exit() is called
- Next value on stack is assumed to be the argument to system()
  - system() will be called with “/bin/bash” !

Return-to-libc attack example

Low Memory High Memory

stack inside main()

Junk Padding	Address of system()	Address of exit()	Pointer to /bin/bash string
	RET		

stack inside system()

	Address of exit()	Pointer to “/bin/bash” string
	RET	argument

system(const char * command)

Shellcode

- What is the malicious code that gets executed once the buffer is overflown?
- It is usually called ***shellcode***, because attackers often choose to execute code that spawns a terminal (shell), which allows them to issue further commands
  - Privilege escalation by exploiting a Set-UID program. How?
- shellcode must be written in assembly language, since it is executed directly on the stack by the CPU
  - shellcode is usually contained in the injected attack buffer
  - A buffer containing shellcode is called a “payload”

What next?

- 64-bit x86 processors have a new function calling convention: the first arguments to a function must be passed in registers instead of being passed on the stack
  - So, attacker can no longer set up a library function call with desired arguments just by manipulating the call stack via a buffer overflow exploit
- Shared libraries also began to remove or restrict library functions useful to an attacker (e.g., system calls wrappers)
- How did attacks evolve?
- Return to oriented programming (ROP) attacks!
  - Attacker uses chunks of library functions, instead of entire functions themselves
  - Functions that contain instruction sequences that pop values from the stack into registers.

Return to oriented programming (ROP)

- The x86 architecture uses a variable-length CISC instruction set, which is very “dense”: any random sequence of bytes is likely to be interpretable as some valid set of x86 instructions
- Method:
  - Search for an opcode (machine language instruction) that alters the control flow, such as the return instruction (0xC3)
  - Look backwards in the binary for preceding bytes that form possibly useful instructions. These instructions form a “gadget”.
  - These sets of instruction "gadgets" can then be chained by overwriting the return address, via a buffer overrun exploit
  - Overwrite the return pointer on stack with the address of the first instruction of the first gadget
  - The first address of subsequent gadgets is then written successively onto the stack

ROP Method - continued

- At the end of the first gadget, a return instruction will be executed, which will pop the address of the second gadget off the stack and jump to it.
- At the conclusion of that gadget, the chain continues with the third, and so on.
- By chaining the small instruction sequences (gadgets), an attacker is able to produce arbitrary program behavior from pre-existing library code.

- Given any sufficiently large quantity of code (such as libc, the C standard library), sufficient gadgets will exist for Turing-complete functionality (see article by H. Shacham, *"The geometry of innocent flesh on the bone: return-into-libc without function calls (on the x86)*, in ACM CCS 2007)
- ROPgadget: An automated tool developed to help automate the process of locating gadgets and constructing an attack against a binary. It searches through a binary looking for potentially useful gadgets, and attempts to assemble them into an attack payload that spawns a shell to accept arbitrary commands from the attacker.

Other developments on this topic?

- Blind Return Oriented Programming (BROP)(research article published in IEEE Security and Privacy Symposium, May 2014)
- The BROP attack makes it possible to write exploits without possessing the target's binary!
- More details at:

  http://www.scs.stanford.edu/brop/

Preventing buffer overflow attacks

Preventing Stack-based Buffer Overflow Attacks

1. Educate programmers about the risks of insecurely copying user-supplied data into fixed-size buffers
   - Ensure that a program never attempts to copy more information than can fit in a buffer

- This problem is specific to C, it cannot happen in Java
- For example, the safer ***strncpy*** function should be used instead of ***strcpy***

Preventing Stack-based Buffer Overflow Attacks

2. The OS can check if a buffer overflow has occurred (and then prevent redirection of control to malicious code)

- Incorporate a ***canary***, a random value that is placed just before the return address
  - The OS regularly checks the integrity of this canary value and if it has been changed, it knows that the buffer has been overflowed

Normal (safe) stack configuration:

Buffer	Other local variables	Canary (random)	Return address	Other data

Buffer overflow attack attempt:

Buffer	Overflow data	Corrupt return address	Attack code

Preventing Stack-based Buffer Overflow Attacks

3. The OS can enforce a no-execution permission on the stack memory segment
   - This prevents executing malicious code that exists on the stack
   - Can be defeated with a return-to-libc attack

4. Use address space layout randomization (ASLR)
   - The data in a process's address space is rearranged at random, making it extremely difficult to predict where to jump in order to execute the malicious code
   - Popular ASLR implementations have been shown to provide an insufficient amount of randomness to fully prevent attacks

Where can you read more about this topic?

- Chapter 3.4 from the Textbook